

Problem Statement and Scope

The goal is to design a service that keeps records in sync between two systems. System A is our internal system, we have full access to its backend and storage. System B is an external system and we can interact with it only through its API.

When a record changes in one system, the change should be reflected in the other. The sync is bidirectional and supports create, update, and delete operations. The system must handle over 300 million sync requests per day, with near real time latency and 99.9% availability.

What this submission covers: The design document describes the syncing of records from System A to External systems. The code implements the A to B direction i.e when a record changes in System A, the sync service receives that change, figures out which external system it needs to go to, converts the data into the format that external expects, and sends it. The reverse direction(B to A) is briefly discussed in the design but not covered in the implementation.

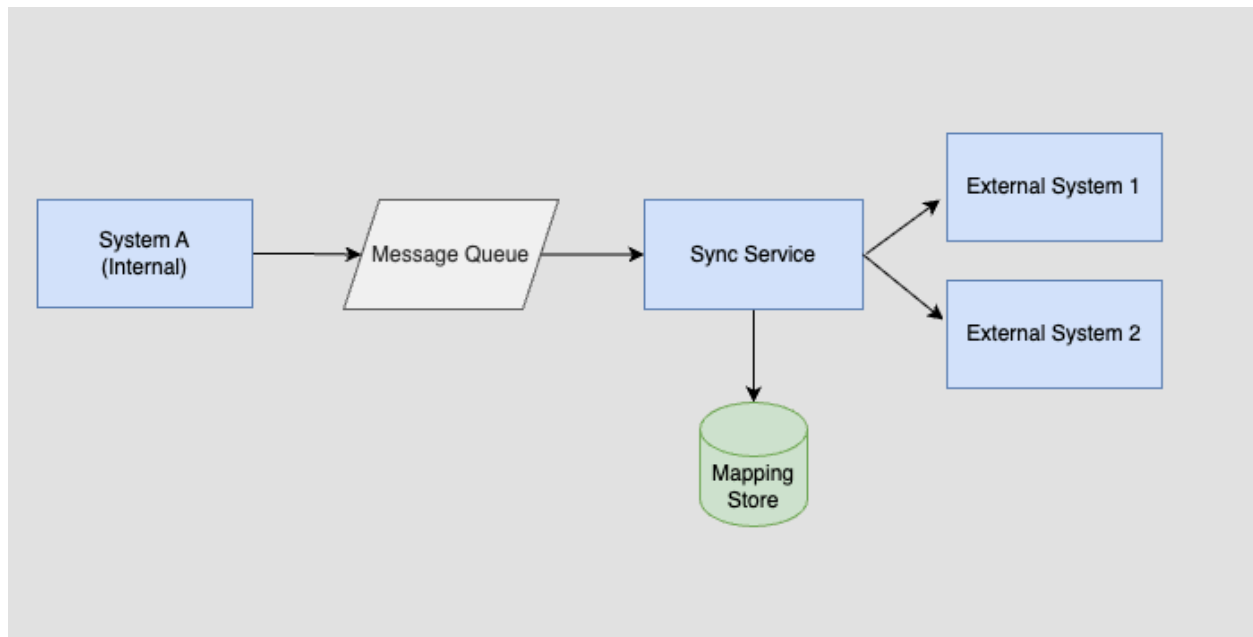
This sub problem was chosen because it covers the core of what the sync service does. It receives a change, converts data between different formats, decides where to send it, and actually delivers it to the right external system.

Assumptions

- When a record changes in System A, the internal system emits an event that contains the full updated record along with what operation happened. Sync service consumes this event.
- Multiple external systems need to be supported. A change in one record in system A can be mapped to a record in multiple external systems. But one record in the internal system maps to only one record in an external system.
- There can be multiple types of records that can be synced. It can be a record with information about a user, it can be a record with information about some order.
- Rules Engine checks if the record needs to be synced or not. If it needs to be synced then it returns the external system the record needs to be synced to.
- Each external system has its own api that supports creating , updating, deleting records. The specific endpoints and their contract differ across systems, but they all support these operations. The api endpoints and their request and response contract is known.

- On creation of a record in the internal system, sync service will keep a record of mapping between internal and external system record ids.
Any update or delete event without a preceding create event, will be rejected by the sync service.

Architecture Overview



The system consists of four main components:

System A is the internal system. When a record is created, updated, or deleted in System A, it emits an event containing the record data, the record type, the operation type, and a source field indicating who caused the change. For deletes, only the record ID and metadata are needed.

Message Queue sits between System A and the sync service. System A publishes the events to the queue, and the sync service consumes from it. The queue decouples the two. System A does not need to wait for the sync service to process the event. If the sync service goes down, events remain in the queue and are processed when it comes back.

Sync Service handles the synchronisation work. It reads change events from the queue, transforms the data, decides which external system(s) the change should go to, converts the data into the format that external system expects, and sends it via API call. It maintains a mapping store, a database that tracks which record in System A corresponds to which record in each external system. This mapping is created the first time a record is created and used for all future operations.

The sync service communicates with the external systems through specific adapters. Each external system gets its own adapter that knows how to talk to that system's API. Adding support for a new external system means writing a new adapter without changing the core sync logic.

External Systems are third party systems accessible only via API. Each has its own data format, rate limits.

Design Considerations

Handling 300 Million Requests Per Day

300 million requests per day translates to roughly 3,500 requests per second on average. The design handles this through horizontal scaling.

The sync service is stateless per event. Each event is processed independently. This means we can run multiple instances of the sync service in parallel, each consuming events from the message queue. To handle more load, more instances can be added.

The message queue is partitioned by record ID received from System A. All events(create, update, delete) for the same record go to the same partition. This allows parallel processing across different records. It also ensures that change events for the same record are processed in order. Without this ordering guarantee, an update followed by a delete for the same record could be processed out of order and leave the external system in a wrong state.

99.9% Availability

The design achieves this through durability at the message queue and replication of the database.

The message queue provides durability. If the sync service goes down, events are not lost. They sit in the queue and are processed when the service recovers.

Running multiple instances of the sync service improves availability as well. If one instance crashes, the message queue rebalances the partitions across the remaining instances and processing continues.

The mapping store uses DynamoDB. Within a region, the table is replicated synchronously across three Availability Zones, so the loss of any single AZ does not affect availability or durability. This is handled by DynamoDB itself and requires no failover logic in the sync service. Without the mapping store, the sync service cannot look up external record IDs, so choosing a storage system with built in multi AZ replication eliminates it as a single point of failure.

Failure Handling: Validation Failures

If inbound validation fails (the event from System A does not match the expected schema), the event is rejected immediately and moved to a validation dead letter queue. No transformation,

rule evaluation, or API calls are attempted. If outbound validation fails (the transformed data does not match the target external system's schema), the sync to that specific external system is skipped. The failed event is moved to the same validation dead letter queue. Syncs to other external systems from the same event are not affected.

Validation failures are not retried automatically because the data itself is bad. Retrying the same event would produce the same failure. These events can be checked to find the root cause, such as a schema change in System A or a missing required field.

Failure Handling: External System Failures

When an API call to an external system fails, the adapter retries with exponential backoff. It waits 1 second before the first retry, then 2 seconds, then 4, and so on. This gives the external system time to recover without overwhelming it with repeated requests.

If the call keeps failing after a configured number of retries, the event is moved to a separate retry dead letter queue. A retry job reads from this queue on a schedule, waits for a configured delay, and resubmits the events back to the main queue for reprocessing. If the event fails again after another round of retries, it goes back to the retry dead letter queue with an incremented retry count. After a maximum number of total attempts, the event is flagged for manual investigation.

The sync service does not block on a failing external system. Events for other external systems and other records continue to process normally. For partial failures where an event needs to sync to two external systems and one succeeds but the other fails, only the failed sync is retried. The successful one is not repeated.

Idempotency

The message queue guarantees at-least-once delivery, which means the same event may be delivered to the sync service more than once. The sync service must handle this without creating duplicate records in the external system.

For the create operations, the sync service checks the mapping store before calling the external system. If a mapping already exists for this System A record ID and the target external system, the record was already created in a previous attempt. The sync service treats it as an update instead and sends the data to the external system. This avoids creating a duplicate record while still ensuring the external system has the latest data.

For update operations, the assumption is that sending the same update more than once results in the same state in the external system. For delete operations, deleting a record that is already deleted either succeeds silently or returns a "not found" error which the adapter handles gracefully.

Sync Service (A to B sync)

The sync service uses an internal model to represent data internally. It is explained in more detail below.

Sync Service Internal Model

System A has its own data format. Each external system also has its own format. Without an intermediate model, we would need a direct mapping from System A to every external system. Every time we add a new external system, we would have to write a new mapping from System A. Every time System A's format changes, we would have to update every mapping.

Sync service internal model is an internal data format owned by the sync service. It helps to solve the problems stated above. Instead of mapping directly between systems, we always go through the internal model.

System A format → Internal Model → External System format

Now adding a new external system required updating only the mapping from the internal model to that system's format. A change in System A's format only requires updating mapping from System A to the internal model. This helps to decouple System A format from external systems.

Sync Overview

When a record changes in System A, the change event flows through the following stages in order:

1. **Sync service receives the event** from the message queue
2. **Echo check** - look at the source field. If the sync service caused this change, discard the event.
3. **Inbound validation** - validate the record coming in from System A against an expected schema.
4. **Inbound transformation** - convert the data from System A's format into the sync service internal model.
5. **Rules evaluation** - determine what sync action to take (create, update, or delete) and which external system to send it to.
6. **Adapter** - transforms the internal model into the target external system's format, validates the outgoing payload against the target system's schema, and sends the data to the external system.

The rest of this section explains each of the key parts in detail.

Inbound Validation

After reading the record from the message queue, the event from System A is validated against System A's expected schema. This catches problems with the incoming data before we attempt to transform it.

Examples of what inbound validation catches:

- A required field is missing (e.g., record has no ID).
- A field has an unexpected type (e.g., a date field that was changed to an integer instead of a string).
- A field has an invalid value (e.g., phone number contains letters).

If inbound validation fails, the event is rejected immediately. This prevents the transformer from crashing or producing bad output due to unexpected input.

Inbound Transformation

If the event passes validation, the inbound transformer converts it from System A's format into the internal model. This is a straightforward field by field mapping.

For example, if System A represents a user record as:

```
{
  "user_id": "U123",
  "first_name": "Rahul",
  "last_name": "Sharma",
  "phone": "9876543210"
}
```

The internal model might represent the same record as:

```
{
  "id": "U123",
  "firstName": "Rahul",
  "lastName": "Sharma",
  "phoneNumber": "+919876543210"
}
```

Rules Engine

Now, the sync service needs to decide if the incoming record should be synced at all, and if so, which external system(s) should receive it. This is the job of the rules engine.

What the Rules Engine Decides

The rules engine takes the event and returns the list of external systems the change should be routed to. If the list is empty, the change is not synced anywhere and processing stops for that event. A single change can be routed to more than one external system. This is how one record in System A fans out to its corresponding records in multiple external systems.

The rules are loaded from a configuration file at startup rather than being hard coded. This is to keep the logic of routing separate from the definition of the rules. The configuration is assumed to be well formed and validating the configuration itself is out of scope for this submission.

Rule Engine holds the list of configured rules and, for each change, asks every rule whether it applies and collects the targets of the ones that do.

Adding new Rules

Rules do not all have to match on the same thing. The common case is routing by record type ("all user records go to External System 1 and External System 2"). But a rule could just as well match on some other field of the record.

If a new rule needs to be added, the engine does not need to change, and existing rule types also remain the same. This keeps the routing layer open to new kinds of rules without modifying what already works.

The output of the rules engine (the set of target external systems) decides which adapters are invoked next.

Adapters

Adapter here is explained within the sync service to make it easier to understand the flow. It can be split into a separate service to handle per external system rate limiting. That is explained in the section - [External System Rate limiting](#)

Once the rules engine determines which external system to sync to, the internal model is handed to the appropriate adapter for each target system. The adapter takes care of transforming the data into the external system's format, validating it, and making the API call.

What Is an Adapter?

An adapter is the component that talks to a specific external system's API. Each external system has its own adapter. The adapter knows the details specific to that system, including its API endpoints, request format, and rate limits. The adapter helps to decouple the external system from the rest of the sync service.

Adapter Components

Each adapter contains:

Outbound transformer: converts the internal model into the format this external system expects. For example, External System 1 might expect:

```
{  
  "first": "Rahul",  
  "last": "Sharma",  
  "contact_phone": "9876543210"  
}
```

While External System 2 might expect:

```
{  
  "fullName": "Rahul Sharma",  
  "phone": "+91-98765-43210"  
}
```

System 1 has separate first/last name fields while System 2 has a combined full name. System 1 wants a boolean for active status while System 2 wants a numeric code. The date and phone formats also differ. Each adapter's outbound transformer handles these differences. The internal model stays the same, and only the outbound mapping changes per external system.

Outbound validation: After the outbound transformer converts the data, the adapter validates the payload against the external system's schema before making the API call. This catches problems specific to the external system. If outbound validation fails, the sync to this external system is rejected, but syncs to other external systems from the same event are not affected.

External System Rate Limiting

Each external system has its own rate limits, and the sync service must respect that. Different external systems will have different rate limits, some allowing thousands of requests per second and others allowing only a few hundred.

To handle this, the sync service can be split into two services after rule evaluation.

Sync Service consumes events from the main message queue and does all steps till rule evaluation. Once the rules engine determines which external systems should receive the record, the pipeline service writes a message per external system to a separate Kafka topic for each external system. That is where this service's job ends. It does not call any external system API.

Adapter Service can be a single service with one codebase that handles outbound transformation, outbound validation, and the external system API call. It is deployed separately for each external system, with each deployment configured to consume from that system's Kafka topic. The same code runs everywhere, just with different configurations. An adapter for a specific kafka topic can be a separate consumer group and responsible for sending records to a specific external system.

Each adapter deployment consumes from its own topic at the rate its external system allows. If External System 1 allows 1,000 requests per second, we run more instances of its adapter worker. If External System 2 allows 50 requests per second, we run fewer instances. The Kafka topic buffers any events that arrive faster than the external system can handle.

If an external system goes down or starts returning rate limit errors (HTTP 429), its adapter needs to take care of retrying with backoff. That system's topic fills up other adapters that are unaffected. When the external system recovers, the adapter workers drain the queue up messages at the allowed rate.

This also simplifies adding a new external system. You write a new adapter configuration (outbound transformer, schema, API client details), create a new Kafka topic, add a routing rule in the pipeline service, and deploy another instance of the adapter worker service with the new configuration.